

面向 Fast-WAM Optional-IDM 的因果风险包络缓存

方法名称: Causal Risk-Envelope Cache (CREC)

研究动机

Fast-WAM Optional-IDM 会反复处理不断到来的观测。问题不只是重复计算：观测变化后，一个缓存单位可能同时影响动作输出和预测视频输出；两个单位即使隐藏状态漂移相近，它们对下游的影响也可能很不一样。单一的局部漂移阈值把所有单位当成可互换的对象，即使少数高风险单位已经承担了大部分动作与视频误差。

这个方案保持现有 Fast-WAM 模型和计算图不变，直接测量这种不对称性。C3ache 提供周期性 World-Action caching 参考，EfficientVLA 提供基于特征相似度的复用参考，WorldCache 和 X-Cache 则说明异质或结构感知的复用可以保持 rollout 质量。就当前 scaffold 所表示的内容而言，这些工作都没有测量让某个缓存单位变旧后产生的动作与视频联合下游风险，也没有用该测量决定哪些依赖必须刷新。因此，问题是：缺少的操作究竟是不是因果风险分配，而不是又一个相似度分数或固定刷新节奏。

近期 caching 工作让这套测量及时可行。C3ache、WorldCache 和 X-Cache 提供了相关复用参考；DriveCache 是一个边界案例，它已经结合了场景级调度、动作感知响应预算，以及由因果漂移触发的刷新或重规划。因此，CREC 不对所有动作感知的预算式 caching 做宽泛主张。这里的有限问题是：Fast-WAM Optional-IDM 是否存在独特的单位级动作/视频联合风险包络；这种异质性是否可以检验；以及在现有 checkpoint 和 native GPU timing 下，是否能够评估覆盖完整后继的失效处理。

如果这个缺口被补上，Fast-WAM caching 就会得到一个明确的分配对象：在 skip rate 固定时，刷新实证上高风险的尾部，复用其余大部分单位，而不再把局部漂移当成充分依据。动作与视频联合误差以及 closed-loop task success 会成为 cache validity boundary 的一部分。通过检验后，它还可以成为一个可复用的因果诊断，用来判断 latency 下降是否具有机制意义；同时它只是补充已有复用机制，并不声称取代它们。

方法

Background

建立现有 Fast-WAM Optional-IDM 计算图，并取得每个单位的干预测量。

1. 从按 episode、时间和 observation 编号的 held-out replay 记录开始。通过 cache-unit registry 读取每个单位稳定的 $'unit_{id}'$ 、产生它的节点、tensor 形状与 dtype、缓存 tensor 和 uncached reference tensor。对一个单位在一个观测变化上的样本，只替换该时间点的缓存 tensor，同时固定相同的 receding observation、模型参数、随机数状态，以及 Fast-WAM Optional Inverse Dynamics Model (IDM) 计算图的其余部分。【作者需决定：缓存单位究竟是完整的中间 tensor、图节点输出，还是更细的 tensor block，并在 replay 前固定 registry 和稳定标识】运行 action head 和 predicted-video head，把 uncached 与 stale 两组输出连同 episode、时间、单位和干预元数据一起保存。对匹配的输出元素计算动作偏差和预测视频偏差，再按观测变化幅度 $'d'$ 分组。【作者需决定：在校准前固定 $'d'$ 的表示、归一化方式、距离或范数，以及分层规则】使用第 1 步（从按 episode）定义的估计量，把加权的配对偏差求条件经验均值，得到 $'r_u(d)'$ ；每行保存 $'unit_{id}'$ 、变化幅度、样本数、两种分量偏差和风险包络估计。【作者需决定：确定动作与预测视频的权重及其来源，包括等权、按任务单位归一化，还是在独立校准集上选择】单位级风险包络把动作分支与预测视频分支的下游偏差，按观测变化幅度联合起来。

$$r_u(d) = \mathbb{E} [\lambda_a \Delta_a + \lambda_v \Delta_v \mid u, d] \quad (1)$$

为什么：这一步提供缺失的因果干预，把局部隐藏状态变化与某个旧缓存单位造成的下游影响分开。

M1_risk_envelope

检验联合下游风险的异质性，并把校准后的风险包络变成刷新分配。

2. 按 $'unit_{id}'$ 和观测变化幅度分组，用与第 1 步（从按 episode）相同的配对偏差估计量重新计算单位级风险包络。构造单位标签置换零分布 (unit-label permutation null)：重新分配完整的单位标签，同时让每个风险值继续绑定自己的变化幅度；每次置换都重新计算选定的异质性统计量。将观测统计量与零分布比较，保存通过/失败门、校准后的风险包络、观测统计量、零分布摘要和判定记录。【作者需决定：预先固定风险包络的校准规则、异质性统计量、判定阈值和置换次数】

为什么：方法需要先证明风险确实存在异质性。没有这道门，global drift rule 与提出的高风险尾部在机制上就无法区分。

3. 载入通过检验的风险包络、cache-unit dependency graph，以及按 $'unit_{id}'$ 编号的单位重算权重表。使用刷新标志 $'z_u'$ ：1 表示刷新，0 表示复用；把第 2 步（按 $'unit_{id}'$...）确定的观测变化幅度范围交给二元选择器。求解第 3 步（载入通过检验的风险包络）定义的选择问题，使复用单位留下的风险低于指定的下游误差上限，同时最小化总重算权重。【作者需决定：确定跨 $'d'$ 的约束方式，是逐幅度约束、按分布加权的汇总，还是最坏情况汇总，并说明

误差上限与匹配 skip rate 如何固定】为每个单位保存 ‘ $unit_{id}$ ’、刷新标志、选定的变化幅度范围、采用的风险值和重算权重，并附上匹配的 skip fraction 与选择器诊断信息。刷新指示变量在满足下游风险预算的同时，最小化重新计算的代价。

$$\min_{\mathbf{z}} \sum_{u=1}^U z_u c_u \quad \text{s.t.} \quad \sum_{u=1}^U (1 - z_u) r_u(d) \leq B, \quad z_u \in \{0, 1\} \quad (2)$$

为什么：这一步把异质性测量变成方法的核心 caching 操作，而不是另一个只描述缓存的分数。

M2_validation

执行覆盖完整后继的刷新，并评估联合误差、*native latency*、*task success* 和置换 control。

4. 把第 3 步（载入通过检验的风险包络）中刷新标志为 1 的单位组成刷新集合。沿 dependency graph 的 producer-to-consumer 边，计算第 4 步（把 S3 中刷新标志为 1...）定义的后继失效集合；清除选中单位及其后继的所有缓存条目，刷新选中单位，再按拓扑顺序重算失效节点，之后才运行两个输出 head。对每个 schedule，使用 episode、观测和随机数状态完全匹配的 uncached rollout 与 control rollout。按每个 episode 记录配对的下游动作均方误差、预测视频均方误差、联合下游误差、同步的 native graphics processing unit (GPU) 延迟和 task success。随后把完整风险包络双射置换到不同的 ‘ $unit_{id}$ ’ 上，用相同的选择器和相同的 skip fraction 重跑，并保存配对汇总、原始分配和置换 control 结果。【作者需决定：定义联合误差如何组合、输出时间范围、episode 汇总方式、task-success 判定，以及同步 native GPU 计时的精确边界】失效集合覆盖所有被刷新单位的图后继，避免旧依赖在刷新后继续被静默复用。

$$\mathcal{I}(\mathcal{R}) = \{v \mid \exists u \in \mathcal{R} : u \prec v\} \quad (3)$$

为什么：重新计算全部后继，可以检验这项分配是否在真实 Fast-WAM 计算图中成立，而不是只改善某个孤立的中间指标。